

Exercícios de Java: Interação entre Objetos

Dani J. Czelusniak, Prof. Dr.

1 Exercício 1: Sistema de Automação Residencial Simples

Enunciado

Desenvolva um sistema básico para controlar uma casa inteligente. Crie uma classe `Lampada` que contenha os atributos `marca` (String), `potencia` (int) e `ligada` (booleano). Crie também uma classe `Interruptor` com os atributos `comodo` (String), `cor` (String) e `lampadaConectada` (do tipo `Lampada`).

A classe `Lampada` deve possuir métodos para `ligar()`, `desligar()` e `exibirEstado()`. A classe `Interruptor` deve possuir os métodos:

1. `conectarLampada(Lampada l)`: Associa o objeto lâmpada ao interruptor.
2. `acionar()`: Se a lâmpada conectada estiver ligada, ele a desliga. Se estiver desligada, ele a liga.

Na classe `Main`, instancie uma lâmpada e um interruptor. Conecte a lâmpada, acione o interruptor algumas vezes e exiba o estado no console.

2 Exercício 2: Sistema de Pet Shop

Enunciado

Crie um sistema que simule um serviço de banho em um Pet Shop. Crie uma classe `Cachorro` com atributos `nome` (String), `raca` (String) e `nivelSujeira` (inteiro de 0 a 10). Crie uma classe `PetShop` com atributos `nome` (String), `caixa` (double) e `precoBanho` (double).

A classe `Cachorro` terá os métodos `brincarNaLama()` (que eleva a sujeira ao máximo) e `exibirStatus()`. A classe `PetShop` terá os métodos `darBanho(Cachorro c)` (que zera a sujeira do cachorro e adiciona o valor do banho ao caixa da loja se o cachorro estiver sujo) e `exibirCaixa()`.

Na classe `Main`, crie um cachorro, faça-o brincar na lama e depois leve-o ao pet shop instanciado para tomar banho. Exiba o caixa final da loja.

3 Exercício 3: Sistema Bancário Básico (Encapsulamento)

Enunciado

Avançando nos conceitos, agora protegeremos nossos dados. Crie uma classe **Cliente** com os atributos **privados** `nome`, `cpf` e `email`. Crie uma classe **ContaBancaria** com os atributos **privados** `numero` (int), `titular` (do tipo **Cliente**) e `saldo` (double).

Crie métodos de acesso (getters) para o nome do cliente. Na conta bancária, crie métodos públicos `depositar(double valor)` e `sacar(double valor)`, garantindo que não se pode sacar mais do que o saldo disponível e não se deposita valores negativos. Crie também um método para `exibirSaldo()`.

No **Main**, instancie o cliente, instancie a conta associando o cliente a ela e realize operações de depósito e saque para testar as restrições.

4 Exercício 4: Simulador de Corrida Simples

Enunciado

Crie uma classe **Motorista** com atributos **privados** `nome`, `cnh` e `anosExperiencia`. Crie uma classe **Carro** com os atributos **privados** `modelo`, `velocidadeMaxima`, `velocidadeAtual` e `motorista` (tipo **Motorista**).

Na classe **Motorista**, crie métodos para retornar o nome e avaliar se o motorista é experiente (mais de 2 anos). Na classe **Carro**, crie métodos públicos:

1. `atribuirMotorista(Motorista m)`
2. `acelerar(int incremento)`: Aumenta a velocidade, não ultrapassando a `velocidadeMaxima`. Só funciona se houver um motorista.
3. `exibirStatus()`: Mostra quem está dirigindo e a velocidade.

No **Main**, teste a aceleração do carro sem motorista, depois atribua o motorista e teste a aceleração até o limite do carro.

5 Exercício 5: Bateria e Dispositivo Móvel

Enunciado

Implemente um sistema para gerenciar o uso de energia de um celular. Crie a classe **Bateria** com os atributos **privados** `capacidadeTotal` (mAh), `cargaAtual` (mAh) e `saudePercentual`. A bateria deve possuir métodos `descarregar(int quantidade)` (retorna booleano se conseguiu descarregar sem zerar), `carregarTotal()` e `getCarga()`.

Crie a classe **Smartphone** com atributos **privados** `marca`, `modelo`, `armazenamento` e `bateria` (do tipo **Bateria**). O smartphone deve ter os métodos `jogar(int horas)` (cada hora gasta 500 mAh, consumindo da bateria), `conectarCarregador()` e `exibirStatus()`.

No **Main**, crie uma bateria de 2000 mAh e um celular, tente jogar até o celular desligar, conecte o carregador e exiba os status.

6 Exercício 6: Sistema de RPG com Itens

Enunciado

Aprofundando a interação, crie a classe `Pocao` com atributos **privados** `tipo` (`String`), `poderCura` (`int`) e `vazia` (`booleano`). Crie métodos `beber()` que retorna os pontos de cura (e marca a poção como vazia) e `isVazia()`.

Crie a classe `Guerreiro` com **privados** `nome`, `vidaAtual`, `vidaMaxima` e `pocaoEquipada` (tipo `Pocao`). Ele deve ter os métodos `equipar(Pocao p)`, `tomarDano(int dano)` e `usarPocao()` que recupera a vida do guerreiro com base no poder da poção, não deixando ultrapassar a vida máxima.

No `Main`, instancie um guerreiro e uma poção, cause dano a ele, equipe a poção, use-a e apresente o resultado final no console.

7 Exercício 7: Sistema de Reservas de Voos

Enunciado

Crie a classe `Passageiro` com os atributos **privados** `nome`, `passaporte` e `assentoReservado` (inicia como "Nenhum"). Crie os métodos `getNome()`, `setAssento(String)` e `exibirInfo()`.

Crie a classe `Voo` com os atributos **privados** `codigo`, `destino`, `vagasDisponiveis` e `precoPassagem`. O voo deve ter os métodos `venderPassagem(Passageiro p, String assento)` (que associa o assento ao passageiro e diminui as vagas, apenas se houver vagas disponíveis) e `statusVoo()`.

No `Main`, crie um voo com apenas 1 vaga disponível. Instancie dois passageiros e tente vender passagem para os dois, demonstrando o comportamento quando o voo lota.

8 Exercício 8: Simulação Agrícola Simples

Enunciado

Vamos estruturar objetos que trabalham em conjunto com a passagem do tempo implícita. Crie a classe `Semente` com atributos **privados** `tipo`, `diasCrescimento` e `regada`. Ela deve ter métodos `receberAgua()` (muda `regada` para `true`), `getTipo()` e `prontaParaColher()` (retorna verdadeiro apenas se a semente estiver regada).

Crie a classe `Agricultor` com **privados** `nome`, `moedas`, `nivelExperiencia` e `sementePlantada` (tipo `Semente`). O agricultor possui os métodos `plantar(Semente s)`, `cuidarPlantacao()` (aciona `receberAgua()` na semente e aumenta a experiência) e `colher()` (se a semente plantada estiver pronta, remove a semente, adiciona 50 moedas e avisa no console).

No `Main`, o agricultor deve plantar uma semente, tentar colher antes de cuidar, e depois cuidar para conseguir colher o fruto.